Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

ALGORITHM: Parent Creates Multiple Children and Synchronizes using waitpid()

1. START
2. Declare an array child[NUM_CHILDREN] to store child PIDs and an integer status.
3. Print a message indicating the parent process has started along with its PID.
4. For $i \leftarrow 0$ to NUM_CHILDREN $- 1$ do
   - 4.1 Call *fork*() and store the return value in child[i].
   - 4.2 If *fork*() < 0 then
   - 4.3 Print error message and terminate the program.
   - 4.4 Else if *fork*() $== 0$ then // Child process
     - 4.4.1 Print message: "Child $i+1$ started" along with child PID.
     - 4.4.2 Sleep for $(i+1) \times 2$ seconds (to create different completion times).
     - 4.4.3 Print message: "Child $i+1$ completed".
     - 4.4.4 Exit the child process with exit status $(i+1)$.
5. End For
6. Print message: Parent waiting for children using waitpid().
7. For $i \leftarrow 0$ to NUM_CHILDREN $- 1$ do
8. Call wait*pid*(child[i], &*status*, 0) and store the return value in *pid*.
9. If wait*pid*() $== -1$ then
10. Print error message and terminate the program.
11. Else
12. If WIFEXITED(*status*) is true then
13. Print: Child with PID *pid* terminated with exit status WEXITSTATUS(*status*).
14. End If
15. End If
16. End For
17. Print message: All child processes completed. Parent exiting.
    STOP

Sample Output:

Parent Process Started
Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521
Child 2 started. PID = 4523, Parent PID = 4521
Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using wait():

Child 1 completed. PID = 4522
wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523
Child 3 completed. PID = 4524
waitpid() detected child PID 4522 finished with exit status 10
waitpid() detected child PID 4523 finished with exit status 11
waitpid() detected child PID 4524 finished with exit status 12

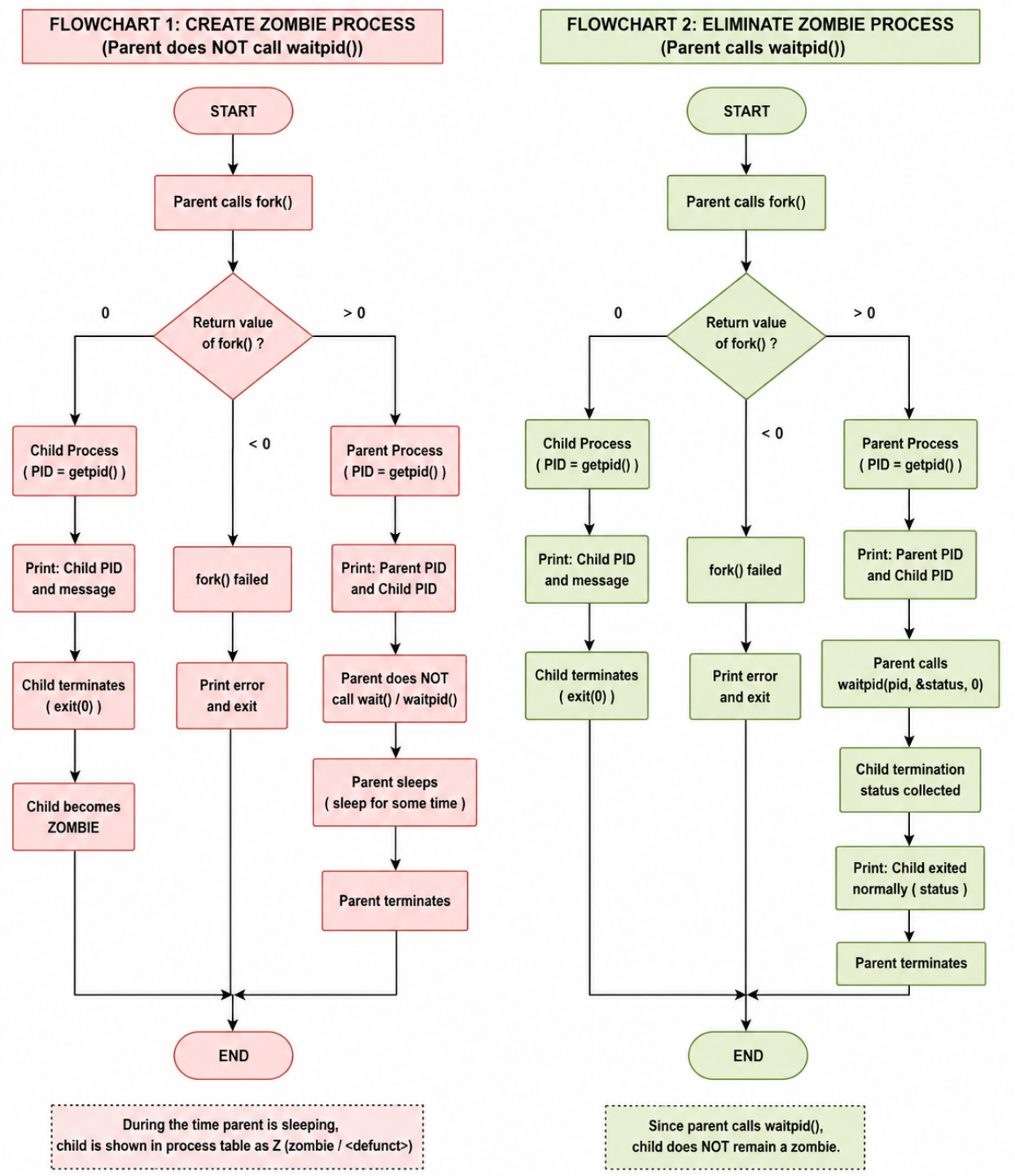
Parent process completed.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
   pid_t pid1, pid2, pid3;
   pid_t wpid;
   int status;

   printf("Parent Process Started\n");
   printf("Parent PID: %d\n", getpid());

   // ---- Create Child 1 ----
   pid1 = fork();
   if (pid1 == 0) {
      printf("Child 1 started. PID = %d, Parent PID = %d\n", getpid(), getppid());
      sleep(1);
      exit(10);   // exit status 10
   }

   // ---- Create Child 2 ----
   pid2 = fork();
   if (pid2 == 0) {
```

```
    printf("Child 2 started. PID = %d, Parent PID = %d\n", getpid(), getppid());
    sleep(2);
    exit(11);   // exit status 11
  }

  // ---- Create Child 3 ----
  pid3 = fork();
  if (pid3 == 0) {
    printf("Child 3 started. PID = %d, Parent PID = %d\n", getpid(), getppid());
    sleep(3);
    exit(12);   // exit status 12
  }

  printf("All 3 child processes have been created.\n");

  // ---- Using wait() to reap Child 1 (first to finish, order not guaranteed) ----
  printf("\nUsing wait():\n");
  wpid = wait(&status);
  if (WIFEXITED(status)) {
    printf("Child completed. PID = %d\n", wpid);
    printf("wait() detected child PID %d finished with exit status %d\n",
        wpid, WEXITSTATUS(status));
  }

  // ---- Using waitpid() to reap remaining children in a specific order ----
  printf("\nUsing waitpid():\n");
  wpid = waitpid(pid2, &status, 0);
  if (WIFEXITED(status))
    printf("waitpid() detected child PID %d finished with exit status %d\n",
        wpid, WEXITSTATUS(status));

  wpid = waitpid(pid3, &status, 0);
  if (WIFEXITED(status))
    printf("waitpid() detected child PID %d finished with exit status %d\n",
        wpid, WEXITSTATUS(status));
  printf("\nParent process completed.\n");
  return 0;
}
```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Flow Chart:



Sample Output:

Parent process: PID = 158
Child process: PID = 159

Child PID = 159
Child process terminating...
Parent sleeping...
Parent terminating...

Ps -el

```
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0     1     0  0  80   0 -  2238 ?     ?        00:00:00 init
0 S     0     8     1  0  80   0 -  2238 ?     tty1     00:00:00 init
0 S  1000     9     8  0  80   0 -  3590 ?     tty1     00:00:00 bash
0 S     0   141     1  0  80   0 -  2238 ?     tty2     00:00:00 init
0 S  1000   142   141  0  80   0 -  3527 ?     tty2     00:00:00 bash
0 S  1000   158     9  0  80   0 -  2680 ?     tty1     00:00:00 week4_T2
0 Z  1000   159   158  0  80   0 -     0 ?     tty1     00:00:00 week4_T2
0 R  1000   160   142  0  80   0 -  4086 ?     tty2     00:00:00 ps
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
   pid_t pid;

   printf("Parent process: PID = %d\n", getpid());

   pid = fork();
```

```
    if (pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);   // Child exits immediately
    } else {
        // Parent process — deliberately does NOT call wait()
        printf("Parent sleeping...\n");
        sleep(30);   // During this time, the child is a ZOMBIE
        printf("Parent terminating...\n");
    }

    return 0;
}
```